

SAS — Alert Engine: Required Changes

What the milestone module now produces, and what the alert engine must evaluate · from Ronaka · 2026-08-16

0. Summary

The milestone builder was extended. Everything below is **definition only** — the builder/library/assignment stores it. The alert engine is what **evaluates** a milestone against a shipment and fires alerts, so these additions need engine support:

- New milestone type **status** — evaluate it exactly like **comparison**.
- New milestone type **custom** — no primary check; its logic is entirely in `extra_logics`.
- **Multi-logic**: any milestone can carry extra check blocks (`extra_logics`) combined with AND/OR (`logic_combine`).
- New due-date basis **after_previous_milestone** — resolve at runtime when the prior milestone completes.
- **document** checks `tracking_field` (not `primary_field`).

1. Where the data lives

Each assigned milestone is one row in **shipment_milestones**. The engine reads these columns:

Column	What it holds
<code>due_date</code>	Deadline (may be null — see §5). Alert rule timing is relative to this.
<code>status</code>	'pending' 'current' 'overdue' 'completed'. Engine updates this.
<code>sequence_order</code>	0-based order within the template. Needed for 'after previous milestone'.
<code>shipment_id</code>	The shipment this milestone is attached to.
<code>milestone_snapshot</code>	jsonb — the full milestone config, frozen at assignment (see §2).
<code>alert_rules_snapshot</code>	jsonb array — the alert rules, frozen at assignment (see §6).
<code>milestone_type</code>	date missing comparison document status custom.

Snapshots are frozen at assignment: editing the library milestone later does NOT change already-assigned shipments. Always read the snapshot, never the live library row.

2. The milestone check (is it satisfied?)

Each milestone reduces to one boolean: is it **satisfied**, or is it still **outstanding** (should alert). The primary check is `milestone_type` + these fields inside `milestone_snapshot`:

Type	Fields	Outstanding (should alert) when
date	<code>primary_field</code>	the field is still null/empty (date not recorded)
missing	<code>primary_field</code>	the field is still null/empty
status	<code>field_a`, `operator`, `fixed_value</code>	the status match holds — evaluate like comparison

Type	Fields	Outstanding (should alert) when
comparison	field_a`, `operator`, `field_b` or `fixed_value`, `threshold_value	the comparison is true
document	tracking_field`, `document_name	tracking_field` is null/empty (doc not uploaded)
custom	(none — see §3)	per extra_logics only

status = comparison. A status milestone is stored as `field_a operator fixed_value` with operator = 'equals' or 'not_equals'. No new logic needed.

Comparison operators depend on Field A's type:

Field A type	Operators
date	missing`, `more_than_x_days_before`, `more_than_x_days_after`, `same_as`, `not_same_as
number	greater_than`, `less_than`, `equal_to`, `not_equal_to
text / status	equals`, `not_equals`, `contains`, `missing

`threshold_value` is the X for the 'more_than_x_days_*' and numeric operators. The right-hand side is `field_b` (another field) OR `fixed_value` (a literal).

3. Multi-logic milestones (extra_logics + logic_combine)

A milestone may bundle extra checks in `milestone_snapshot.extra_logics` — a JSON array of blocks. Each block is { type, ...fields } using the same field names as §2:

```
"extra_logics": [
{ "type": "status", "field_a": "pickup_date_status",
"operator": "equals", "fixed_value": "Completed" },
{ "type": "document", "document_name": "Bill of Lading",
"tracking_field": "job_docs_last_edit_time" }
]
```

`logic_combine` is 'and' or 'or'. The milestone's combined result is the primary check combined with every extra block:

■ **and** → satisfied only when the primary check AND all extra blocks are satisfied.

■ **or** → satisfied when any of them is satisfied.

For fire conditions that gate on the milestone still being outstanding (§6), evaluate the **combined** result, not just the primary check. **custom** type has no primary check — evaluate the `extra_logics` blocks alone (at least one is always present).

4. Resolving a field's value

Use `resolve_field_value(shipment, milestone_key, api_field)` in `services/field_registry.py`. It reads, in order:

- exact sub-key inside `shipments.milestones[milestone_key]`,
- a normalized/alias match inside that group (case/separator-insensitive),
- fallback to a plain shipment column `shipment.get(api_field)` — this is how manually-maintained columns like `estimated_arrival` resolve without a registry row.

`milestone_key` is in the snapshot. Every field a check references (`primary_field`, `field_a`, `field_b`, `tracking_field`) resolves this way — including fields inside `extra_logics` blocks. Door 2 already registers those fields so the sync collects them.

5. Due date computation

`expected_date_source` sets the deadline that rule timing is measured against:

<code>expected_date_source</code>	Due date
<code>another_field</code>	<code>expected_date_field` value + `expected_date_offset` days</code>
<code>days_after_creation</code>	<code>shipment `created_at` + `expected_date_offset` days</code>
<code>after_previous_milestone</code>	previous milestone's <code>`completed_date` + `expected_date_offset`</code> — RUNTIME
<code>manual</code>	set by the admin at assignment
<code>self` (legacy only)</code>	the watched field's own value (no longer offered in the builder)

Important: `_compute_due_date()` resolves everything at assignment EXCEPT `after_previous_milestone`, which it leaves null. The engine must set it at runtime: when milestone N (by `sequence_order`) completes, set milestone N+1's `due_date = previous.completed_date + expected_date_offset`.

6. Alert rule evaluation

`alert_rules_snapshot` is an array; each rule fires independently. Fields:

Field	Values / meaning
timing	'before' 'on_date' 'after' — relative to <code>due_date</code> , with <code>`days_offset`</code> and <code>`fire_time`</code> (HH:MM)
condition	'always' 'if_not_recorded' 'if_comparison_true' 'if_missing' (see below)
recurrence_type	'once' 'daily' 'weekly' 'custom_interval' (every <code>`recurrence_interval`</code> days)
recurrence_end_type	'after_n_times' (<code>`recurrence_end_n`</code>) 'on_date' (<code>`recurrence_end_date`</code>) 'when_condition_met' 'never'
stop condition	<code>stop_condition_field` + `stop_condition_type` ('is_not_null' 'is_null' 'changed' 'equals') + `stop_condition_value</code>
recipient_type	'operations' 'sales' 'consignee' 'custom' (<code>`custom_email`</code>)

Fire conditions — a rule only sends if its condition is true at fire time:

condition	Fires only if
always	unconditionally (a pure time-based reminder)
if_not_recorded	the milestone's watched field is still empty (combined result outstanding)
if_comparison_true	the comparison / status match is still true
if_missing	the required field is still empty

For multi-logic milestones the non-'always' conditions test the **combined** outstanding result (§3). Recipient resolution: operations/sales = the user assigned to the shipment; consignee = the consignee email on the shipment; custom = `custom_email`.

7. Implementation checklist

- Evaluate a single check for all 6 types (status = comparison; document = tracking_field null-check).
- Combine extra_logics with `logic_combine` (AND/OR); custom = blocks only.

- Resolve every field via `resolve_field_value` using the snapshot's `milestone_key`.
- Compute `after_previous_milestone` due dates at runtime from the prior milestone's completion.
- For each rule: check timing vs `due_date`, apply the fire condition, honor recurrence + stop condition, resolve recipient, send.
- Update `shipment_milestones.status` as checks pass / deadlines pass.

8. Edge cases & notes

- A milestone with no due date (e.g. a document milestone whose basis is unresolved) can only run 'on_date'/absolute rules — 'before/after' has no anchor.
- The combine is milestone-wide (all blocks one way). There is no per-block AND/OR grouping and no stage-gating (blocks are evaluated together, not in sequence).
- Nothing here writes to `shipments.milestones` — only the CargoWise sync does. The engine only reads.
- Migrations that must be applied first: `milestone_multi_logic.sql` (`extra_logics` / `logic_combine` columns).